

INFORME DE DEFENSA FINAL

Proyecto: Pico Botella (PB) — Android / Kotlin

Preparación para la Evaluación del Miniproyecto 1

Repositorio: <https://github.com/StiviMoon/PB>

Este documento consolida el resumen técnico general de la aplicación, el detalle exhaustivo de las Historias de Usuario asignadas, la evidencia de participación en GitHub y una guía de posibles preguntas y respuestas para la defensa ante el profesor.

1. Resumen Técnico General de la App

La aplicación sigue el patrón de arquitectura MVVM (Model-View-ViewModel), utilizando SQLite mediante Room y consumiendo una API externa de Pokémon.

Tecnología	Propósito en el Proyecto
Kotlin	Lenguaje de programación nativo
MVVM	Patrón de arquitectura para separar la lógica de la vista
Room (SQLite)	Base de datos local para guardar y eliminar retos
Kotlin Coroutines	Ejecutar operaciones pesadas sin congelar la UI
StateFlow	Manejo de estado reactivo (la UI se actualiza sola al borrar)
Navigation	Navegación entre Splash, Home, Retos e Instrucciones

2. Evidencia de Participación Individual (RA-3)

A continuación se detallan los commits principales que demuestran la contribución directa al repositorio (Usuarios en Git: Kevinseya / kevinseya17):

Hash	Mensaje del Commit	Qué decirle al Profesor
eb4049b	Merge pull request #7 from kevinseya17/feature/eliminar-retos	Profesor, aquí integré mi rama de trabajo al proyecto principal mediante un Pull Request.
8a77e9a	Resolve merge conflicts and integrate upstream changes	Este commit fue importante porque resolví los conflictos al unir mi código con el de la rama principal.
db72d80	Merge HU-6, HU-7 con HU-9 adaptado a nueva BD	Aquí adapté mi código de Eliminar Reto para que funcionara con la tabla Room de la nueva BD.
d143261	fix(home): Mostrar texto del reto al azar	Aquí implementé la lógica para mostrar el texto aleatorio de la HU 12 (Mostrar Reto al azar).
952f25b	feat(retos): Implementar funcion de eliminar reto	Este fue mi primer gran commit donde desarrollé el cuadro de diálogo y lógica de eliminar retos.

Gestión en Jira: Las historias asignadas a mi nombre fueron la **HU 9.0** (Eliminar reto) y la **HU 12** (Mostrar reto aleatorio), las cuales fueron movidas a la columna 'Done / Hecho' durante el sprint.

3. Detalle de Contribución: HU 9.0 y HU 12

HU 9.0 — Eliminar Reto (Cuadro de confirmación)

Muestra un diálogo de confirmación cuando el usuario quiere borrar un reto de la lista.

- **Diseño (RA-2):** Se creó ``dialog_delete_reto.xml`` con fondo blanco (usando ``bg_dialog_white_rounded.xml``), título negro en negrita, la descripción dinámica de la BD, y dos botones de texto 'NO' y 'SI' en color naranja (#FF6F00).
- **Bloqueo del Diálogo:** Se utilizó ``dialog.setCancelable(false)`` en Kotlin para que el diálogo NO se cierre si el usuario toca por fuera o presiona el botón de atrás, cumpliendo el criterio estricto.
- **Lógica de Eliminación (RA-1):** Al presionar 'SI', se llama a ``viewModel.eliminarReto(reto)`` que ejecuta una Corrutina (``viewModelScope.launch``) llamando al ``delete`` del DAO de Room. Gracias al ``Flow``, la lista se actualiza sola.

HU 12 — Mostrar Reto Aleatorio

Tras girar la botella, muestra un diálogo con un reto aleatorio de la base de datos y un Pokémon al azar.

- **Diseño Avanzado (RA-2):** Se usó ``ConstraintLayout`` para lograr que el círculo del Pokémon y el botón naranja 'Cerrar' sobresalgan del fondo negro semitransparente. Esto se logró anclando el constraint top y bottom de la vista al mismo borde.
- **Consulta Aleatoria (RA-1):** Se agregó al DAO la consulta SQL: ``SELECT * FROM retos ORDER BY RANDOM() LIMIT 1``.
- **Consumo de API (RA-1):** Se creó ``PokemonRepository.kt`` usando ``HttpURLConnection`` para descargar el JSON de la Pokédex desde GitHub. La petición se ejecuta en ``Dispatchers.IO`` para no bloquear el hilo principal.

4. Q&A;; Posibles Preguntas del Profesor (Simulación de Defensa)

Pregunta del Profesor: ¿Por qué usaron MVVM y no MVC en este proyecto de Android?

Respuesta: Usamos MVVM para separar la lógica de negocio de la interfaz visual. Al usar ``ViewModel``, los datos sobreviven a los cambios de configuración (como cuando se gira el teléfono). En MVC, el Activity/Fragment se sobrecarga de código y los datos se pierden al rotar la pantalla.

Pregunta del Profesor: Muéstrame cómo hicieron para que la app no se congele al hacer consultas a la Base de Datos o a la API de Pokémon.

Respuesta: Utilizamos Kotlin Coroutines. Para las llamadas a Room usamos ``viewModelScope.launch``, y para la API de Pokémon usamos explícitamente ``withContext(Dispatchers.IO)``. Esto asegura que las operaciones pesadas de lectura, escritura y red ocurran en hilos secundarios, dejando el Main Thread libre para que la UI sea fluida.

Pregunta del Profesor: ¿Por qué eligieron usar Room en lugar de SQLiteOpenHelper estándar?

Respuesta: Room es la recomendación oficial de Google (Jetpack). Nos proporciona verificación de consultas SQL en tiempo de compilación (evita errores en ejecución). Además, Room se integra perfectamente con ``Flow`` y Corrutinas, lo que nos permite tener una base de datos reactiva que avisa a la UI cuando hay cambios (como en la HU 9 al eliminar).

Pregunta del Profesor: En la HU 12, el botón de Cerrar y el ícono del Pokémon se ven por fuera de la caja negra. ¿Cómo programaste eso en la interfaz?

Respuesta: Utilicé las restricciones de ``ConstraintLayout``. Para que el botón sobresalga por abajo, vinculé su ``layout_constraintTop_toBottomOf`` y su ``layout_constraintBottom_toBottomOf`` ambos al borde inferior del contenedor oscuro. Eso hace que el centro del botón se alinee exactamente con el borde del diálogo.

Pregunta del Profesor: Veo que usaron `StateFlow` en el ViewModel de retos. ¿Qué ventaja tiene sobre `LiveData`?

Respuesta: `StateFlow` está construido sobre Corrutinas nativas de Kotlin, lo que lo hace más moderno y no depende de la plataforma Android. Siempre tiene un estado inicial, lo cual es ideal para manejar estados como Loading, Empty y Success en la lista de retos.

Pregunta del Profesor: ¿Cómo lograste en la HU 9.0 que el diálogo no se cerrara si el usuario tocaba la pantalla por fuera?

Respuesta: Instancié un objeto `android.app.Dialog` y llamé a la propiedad `dialog.setCancelable(false)`. Esto bloquea los toques exteriores y también el uso del botón físico/gesto de 'atrás' del celular, obligando al usuario a elegir entre los botones 'SI' o 'NO' tal como pedía el requerimiento de la Historia de Usuario.

Pregunta del Profesor: ¿Cómo descargaste la imagen del Pokémon de internet? ¿Usaste alguna librería?

Respuesta: No utilizamos librerías como Retrofit o Glide para mantener el proyecto con las librerías base permitidas. Utilicé `URLConnection` nativo de Java/Kotlin. Hago un GET a la URL del JSON, extraigo un Pokémon al azar usando `JSONObject`, y luego abro otra conexión para descargar la imagen directamente a un `Bitmap` que finalmente coloco en el `ImageView`.